

# Segmentación de Imágenes SEM mediante AutoML: Comparación de Estrategias de Segmentación

Autor

**Resumen**—Se presenta un enfoque de aprendizaje automático para la detección de zonas frágiles y dúctiles en imágenes obtenidas mediante microscopía electrónica de barrido. El estudio compara modelos del estado del arte basados en transformers con estrategias de segmentación propuestas, integradas en un framework AutoML. La selección del modelo se realiza a partir de métricas de desempeño relevantes, considerando explícitamente las limitaciones computacionales del entorno de ejecución. Los resultados muestran la viabilidad del enfoque bajo recursos restringidos y evidencian el impacto del uso de aumentación de datos y transferencia de aprendizaje en la calidad de la segmentación.

## I. INTRODUCCIÓN

La caracterización de los mecanismos de fractura en materiales es un problema central en la ciencia e ingeniería de materiales, debido a su impacto directo en la evaluación del desempeño mecánico y la confiabilidad estructural. En particular, la identificación de zonas asociadas a comportamientos frágiles y dúctiles a partir de microestructuras proporciona información clave para el análisis del fallo y el diseño de materiales.

Las imágenes obtenidas mediante microscopía electrónica de barrido (SEM) constituyen una de las principales herramientas para el estudio de superficies de fractura y microestructuras. No obstante, el análisis tradicional de este tipo de imágenes se basa en inspección visual experta, lo cual introduce subjetividad, limita la reproducibilidad de los resultados y dificulta su aplicación a grandes volúmenes de datos.

En este contexto, las técnicas de aprendizaje automático y, en particular, los modelos de visión por computador han mostrado un potencial significativo para automatizar tareas de clasificación y segmentación en imágenes complejas. Recientemente, modelos basados en transformers, como Vision Transformer (ViT) y Swin Transformer, han alcanzado resultados destacados en diversas tareas de análisis de imágenes, posicionándose como referentes del estado del arte. Sin embargo, su aplicación práctica suele estar condicionada por la disponibilidad de datos anotados y recursos computacionales elevados.

Estas limitaciones motivan la exploración de estrategias alternativas que permitan aprovechar modelos de alto desempeño bajo restricciones de cómputo y datos. En este trabajo se consideran tanto modelos del estado del arte como enfoques propuestos basados en la descomposición espacial de la imagen y el uso de clasificadores preentrenados, con el objetivo de transferir conocimiento desde tareas de clasificación hacia problemas de segmentación.

Adicionalmente, se adopta un enfoque AutoML para automatizar la selección de modelos y el ajuste de hiperparámetros, empleando metaheurísticas y considerando explícitamente las restricciones impuestas por un entorno de cómputo limitado. El objetivo de este estudio es evaluar la viabilidad y el desempeño de estas estrategias para la detección de zonas frágiles y dúctiles en imágenes SEM, comparando modelos del estado del arte con enfoques alternativos bajo condiciones realistas de entrenamiento y evaluación.

## II. METODOLOGÍA BASADA EN AUTOML

La resolución del problema de segmentación se abordó mediante un enfoque de AutoML diseñado específicamente para garantizar modularidad, reproducibilidad y comparabilidad sistemática entre distintas configuraciones experimentales. La metodología no se limita a la automatización del entrenamiento, sino que establece una arquitectura formal que define claramente cada etapa del flujo de datos y permite intercambiar componentes de manera controlada.

Arquitectura general del pipeline

El núcleo metodológico se basa en un pipeline lineal de procesamiento, en el cual los datos fluyen secuencialmente a través de nodos bien definidos. Este pipeline sigue el esquema general:

Dataset inicial → Nodo de aumentación → Nodo de modelo  
→ Nodo de evaluación → Resultados finales (1)

Cada etapa del pipeline encapsula una responsabilidad única y explícita, evitando dependencias implícitas y facilitando el análisis aislado del impacto de cada decisión metodológica. La coordinación de este flujo está a cargo de una clase orquestadora central, denominada *AutoML*, cuya función es estrictamente organizativa.

El diseño del framework se fundamenta en el uso extensivo de interfaces abstractas. Todas las etapas del pipeline implementan contratos formales definidos a nivel de interfaz, lo que permite desacoplar completamente la lógica de alto nivel de las implementaciones concretas. Este enfoque garantiza que diferentes estrategias —por ejemplo, distintos esquemas de validación o arquitecturas de red— puedan sustituirse sin alterar la estructura global del sistema.

La metodología adopta el principio de composición: la clase *AutoML* no implementa lógica de entrenamiento, aumentación ni evaluación, sino que recibe instancias de nodos que cumplen interfaces predefinidas y las ejecuta en el orden correcto. Esto resulta en un núcleo extremadamente estable y fácil de auditar desde el punto de vista metodológico.

## II-A. Nodo de aumentación y partición de datos

El primer componente del pipeline es el nodo de aumentación de datos, responsable de transformar el dataset original y generar las particiones de entrenamiento y validación. Este nodo recibe el dataset completo y produce una lista de pares  $(D_{train}, D_{val})$  lo que permite soportar tanto divisiones simples como esquemas de validación cruzada k-fold.

Desde el punto de vista metodológico, esta decisión es clave: todas las transformaciones de datos y estrategias de partición quedan formalizadas en un único punto del pipeline, evitando fugas de información y asegurando que todos los modelos evaluados operen bajo condiciones comparables.

## II-B. Nodo de modelo

El nodo de modelo constituye el núcleo computacional del sistema. Para cada par de datasets generado por el nodo de aumentación, este componente entrena una instancia del modelo especificado y posteriormente genera predicciones sobre el conjunto de validación. El resultado de esta etapa es una colección estructurada de pares de máscaras predichas y reales, organizada por pliegue cuando corresponde.

Cada arquitectura se encapsula en una implementación concreta de este nodo, manteniendo la misma interfaz externa. Esto permite comparar modelos de distinta naturaleza bajo un marco experimental idéntico, aislando el efecto de la arquitectura del resto del pipeline.

## II-C. Nodo de evaluación

El nodo de evaluación recibe exclusivamente las predicciones y las máscaras reales producidas por el nodo de modelo. Su responsabilidad es calcular las métricas de desempeño de forma agregada y consistente. Internamente, este nodo delega el cálculo a evaluadores individuales, cada uno asociado a una métrica específica.

Desde el punto de vista metodológico, este desacoplamiento garantiza que las métricas no influyan en el proceso de entrenamiento ni en la generación de predicciones, evitando sesgos y permitiendo incorporar nuevas métricas sin modificar etapas previas del pipeline.

## II-D. Orquestación y ejecución automática

La clase *AutoML* actúa como el punto único de ejecución del experimento. Su método principal ejecuta secuencialmente los nodos de aumentación, modelo y evaluación, y retorna un conjunto final de métricas. Este diseño minimalista reduce la complejidad cognitiva del sistema y asegura que el flujo experimental sea explícito y reproducible.

La automatización se logra mediante la ejecución repetida del pipeline con distintas combinaciones de nodos y configuraciones. De este modo, el proceso de selección de modelos e hiperparámetros se formaliza como una exploración sistemática del espacio de configuraciones, donde el criterio de selección se basa exclusivamente en las métricas producidas por el nodo evaluador.

La construcción concreta de los nodos no se realiza directamente en los scripts experimentales, sino mediante funciones

de configuración ubicadas en un módulo dedicado. Este enfoque separa claramente la definición del experimento de la lógica interna del framework, mejorando la legibilidad, reduciendo errores de configuración y facilitando la reproducibilidad de los resultados.

Esta metodología basada en *AutoML* establece una base sólida y extensible para todo el desarrollo posterior, dentro de un marco experimental unificado y controlado.

## III. DESCRIPCIÓN DEL DATASET

### IV. ESTRATEGIAS DE SEGMENTACIÓN

La segmentación de zonas frágiles y dúctiles se aborda mediante un conjunto de estrategias complementarias que responden a distintos niveles de complejidad y granularidad en la representación de la imagen. En primer lugar, se consideran arquitecturas basadas en Vision Transformers y Swin Transformers, seleccionadas por su capacidad para modelar dependencias espaciales de largo alcance y por su consolidación como enfoques de referencia en tareas de segmentación densa. Estos modelos permiten aprender representaciones globales y jerárquicas directamente a partir de la imagen completa, proporcionando una base sólida para la identificación precisa de regiones con comportamiento mecánico diferenciado.

De forma complementaria, se exploran estrategias de segmentación indirecta que reutilizan conocimiento previamente adquirido en tareas de clasificación. En este enfoque, la información semántica extraída por modelos entrenados para discriminar regiones frágiles y dúctiles se emplea como guía para definir esquemas de segmentación basados en particiones espaciales adaptativas. En particular, se consideran métodos que descomponen la imagen en regiones locales cuya resolución y extensión se ajustan dinámicamente en función de la complejidad visual y la confianza de la clasificación, permitiendo una transición controlada entre análisis global y local.

Este planteamiento unifica modelos de segmentación directa y estrategias guiadas por clasificación bajo un mismo marco conceptual, facilitando la comparación entre enfoques de distinta naturaleza y sentando las bases para la incorporación progresiva de métodos alternativos de particionado espacial en subsecciones posteriores.

### IV-A. Vision Transformer (ViT)

La estrategia de segmentación basada en *Vision Transformer* (ViT) se diseñó siguiendo un enfoque modular que separa explícitamente la definición arquitectónica del modelo del proceso de entrenamiento y evaluación. Esta decisión metodológica permite desacoplar los aspectos conceptuales del modelo de los detalles operativos, facilitando su integración en el framework de *AutoML* y asegurando comparabilidad con otras estrategias de segmentación.

#### IV-A1. Arquitectura encoder-decoder

El modelo adopta una arquitectura de tipo *encoder-decoder*, donde el encoder se basa en un Transformer y el decoder en una red neuronal convolucional. Esta combinación permite explotar la capacidad del mecanismo de autoatención para

capturar dependencias globales, mientras que el decoder convolucional reconstruye la información espacial necesaria para una segmentación densa a nivel de píxel.

El encoder transforma la imagen de entrada en una secuencia de representaciones latentes mediante un proceso de partición en parches no solapados. Cada parche es proyectado a un espacio de alta dimensionalidad, generando una secuencia de *embeddings* que reemplaza la representación basada en píxeles. Dado que los Transformers carecen de noción explícita de orden espacial, se incorporan codificaciones posicionales aprendibles, las cuales preservan la información relativa a la localización de cada parche en la imagen original.

Posteriormente, esta secuencia es procesada por múltiples capas de autoatención, donde cada parche puede intercambiar información con todos los demás. Este mecanismo permite construir representaciones con contexto global, capturando relaciones de largo alcance.

#### IV-A2. Decoder convolucional y reconstrucción espacial

La salida del encoder, expresada como una secuencia de vectores latentes, es reestructurada nuevamente en forma de mapa bidimensional de características. A partir de esta representación de baja resolución espacial, el decoder convolucional realiza una reconstrucción progresiva hasta alcanzar la resolución original de la imagen.

Este proceso se lleva a cabo mediante una serie de etapas de refinamiento y reescalado, en las cuales se combinan convoluciones para la extracción local de características con operaciones de interpolación para el aumento gradual de la resolución. En la etapa final, una proyección lineal por píxel permite obtener, para cada clase, un mapa de activación que representa la pertenencia de cada píxel a las distintas regiones de interés.

#### IV-A3. Integración en el pipeline de AutoML

La arquitectura ViT descrita se integra en el pipeline de AutoML mediante un componente que encapsula tanto el proceso de entrenamiento como la generación de predicciones. Este componente actúa como intermediario entre el modelo de segmentación y el resto del framework, garantizando que el ViT pueda ser tratado de manera homogénea junto con otras arquitecturas.

Durante el entrenamiento, se emplea un esquema supervisado estándar para segmentación multiclase, utilizando funciones de pérdida adecuadas para este tipo de problema y validación sobre datos no vistos para monitorear el desempeño. En la fase de inferencia, las salidas continuas del modelo son convertidas en máscaras discretas mediante una asignación por máxima activación, produciendo mapas de segmentación directamente comparables con las máscaras de referencia.

Esta estrategia combina la capacidad de los Vision Transformers para modelar contexto global con la precisión espacial de decodificadores convolucionales, resultando adecuada para problemas de segmentación.

### IV-B. Swin Transformer

El segmentador basado en *Swin Transformer* se desarrolló siguiendo el mismo principio aplicado al modelo ViT: una separación estricta entre la arquitectura del modelo y la lógica

de entrenamiento y evaluación. Esta coherencia de diseño garantiza modularidad, facilita la comparación experimental y permite integrar el modelo de manera transparente dentro del framework de AutoML.

#### IV-B1. Arquitectura encoder-decoder

Al igual que en el caso del ViT, el modelo adopta una arquitectura de tipo *encoder-decoder*. Sin embargo, el encoder se basa en el Swin Transformer, una evolución de los Transformers para visión computacional que introduce propiedades jerárquicas y de localidad, tradicionalmente asociadas a las redes convolucionales.

El encoder genera representaciones jerárquicas a múltiples escalas espaciales. A partir de la imagen de entrada, el modelo construye progresivamente un conjunto de mapas de características con resoluciones decrecientes, lo que permite capturar información tanto local como global. Este enfoque multiescala resulta especialmente adecuado para tareas de segmentación, donde coexisten detalles finos y estructuras de mayor tamaño.

El mecanismo central del Swin Transformer es la autoatención restringida a ventanas locales, lo que reduce significativamente la complejidad computacional en comparación con la autoatención global. Para evitar la pérdida de información entre regiones disjuntas, estas ventanas se desplazan de manera alternada entre capas consecutivas, permitiendo que la información fluya entre distintas zonas de la imagen. De este modo, el modelo combina eficiencia computacional con una capacidad efectiva de modelar dependencias de largo alcance.

Adicionalmente, la arquitectura fue adaptada para operar sobre imágenes en escala de grises, lo que asegura compatibilidad con el tipo de datos empleado en este trabajo sin alterar los principios fundamentales del modelo.

#### IV-B2. Adaptación de resolución y decoder compartido

El encoder basado en Swin Transformer produce su representación final a una resolución espacial inferior a la utilizada por el decoder convolucional. Para resolver esta discrepancia, se introduce un módulo de adaptación intermedio que reescala las características extraídas por el encoder hasta la resolución esperada por el decoder.

Una vez ajustada la resolución, las características son procesadas por el mismo decoder convolucional utilizado en el modelo basado en ViT. Este decoder realiza una reconstrucción progresiva de la segmentación mediante etapas sucesivas de refinamiento y aumento de resolución, culminando en un mapa de segmentación denso a nivel de píxel. El uso de un decoder compartido evita la duplicación de componentes, mejora la mantenibilidad del sistema y asegura que las diferencias observadas en los resultados se deban principalmente al encoder y no a variaciones en la fase de reconstrucción.

#### IV-B3. Entrenamiento, regularización y evaluación

La integración del modelo Swin en el pipeline de AutoML se realiza a través de un componente que encapsula el proceso completo de entrenamiento y evaluación, manteniendo la misma interfaz que otros segmentadores. Esto garantiza la interoperabilidad dentro del framework y permite su comparación directa con enfoques alternativos.

Un aspecto distintivo de esta estrategia es la incorporación de un mecanismo de *early stopping* durante el entrenamiento.

Este mecanismo monitoriza el desempeño sobre el conjunto de validación y detiene el proceso de optimización cuando no se observan mejoras durante un número predefinido de iteraciones consecutivas. De este modo, se reduce el riesgo de sobreajuste y se selecciona automáticamente el estado del modelo con mejor capacidad de generalización.

En la fase de inferencia, el modelo genera mapas de activación multiclase que son transformados en máscaras de segmentación discretas mediante una asignación por máxima respuesta. Estas máscaras se utilizan posteriormente en el módulo de evaluación, asegurando consistencia con el resto de modelos analizados.

El segmentador basado en Swin Transformer combina eficiencia computacional, representación jerárquica multiescala y mecanismos de regularización explícitos, constituyendo una alternativa de alto nivel dentro del estado del arte para segmentación semántica en imágenes complejas.

#### IV-C. Descomposición quadtree

El enfoque de segmentación basado en quadtree difiere de manera fundamental de los modelos neuronales de tipo extremo a extremo. En lugar de aprender directamente una correspondencia píxel a píxel, este método se formula como un algoritmo recursivo de tipo *divide y vencerás*, que combina un modelo de clasificación independiente con una estrategia adaptativa de partición espacial. Esta aproximación permite razonar sobre regiones completas de la imagen y ajustar dinámicamente el nivel de detalle de la segmentación en función de la complejidad local.

##### IV-C1. Principio de divide y vencerás mediante quadtree

El algoritmo opera de manera recursiva sobre regiones rectangulares de la imagen. Inicialmente, la imagen completa se considera como una única región. Para cada región analizada, se consulta a un clasificador externo con el fin de estimar la clase predominante y un nivel de confianza asociado a dicha predicción.

Si la confianza supera un umbral predefinido, se asume que la región es suficientemente homogénea y se asigna la clase predicha a todos los píxeles que la componen. En caso contrario, el algoritmo interpreta que la región contiene información heterogénea o ambigua y procede a subdividirla en cuatro cuadrantes de igual tamaño. Este proceso se repite de forma independiente para cada subregión.

La recursión se detiene cuando se cumple alguna de las siguientes condiciones: la confianza del clasificador es suficiente, la región alcanza un tamaño mínimo que impide una subdivisión significativa, o se llega a una profundidad máxima de recursión. Como resultado, el método genera regiones extensas y uniformes en zonas simples de la imagen, y particiones más finas en áreas con mayor complejidad estructural.

##### IV-C2. Separación entre segmentación y clasificación

Un rasgo central de este enfoque es la separación explícita entre la lógica de segmentación y el modelo de clasificación utilizado para tomar decisiones locales. El algoritmo quadtree se limita a gestionar la recursión y la asignación espacial de etiquetas, mientras que el clasificador actúa como un

componente intercambiable encargado de evaluar regiones de la imagen.

Esta separación permite desacoplar completamente la estrategia de partición del método concreto de clasificación empleado. En consecuencia, es posible evaluar distintos clasificadores bajo un mismo esquema de segmentación sin modificar el algoritmo quadtree, lo que aporta flexibilidad experimental y facilita el análisis comparativo de modelos.

##### IV-C3. Proceso de ajuste de parámetros mediante meta-heurísticas

A diferencia de los modelos neuronales clásicos, el proceso de entrenamiento de este segmentador no se basa en retro-propagación de gradientes. En su lugar, el ajuste se centra en la optimización de los hiperparámetros que controlan el comportamiento del algoritmo recursivo, tales como el umbral de confianza, el tamaño mínimo de región y la profundidad máxima de subdivisión.

Este ajuste se plantea como un problema de optimización global y se resuelve mediante una metaheurística de recocido simulado (*simulated annealing*). El procedimiento explora el espacio de hiperparámetros evaluando configuraciones candidatas sobre un conjunto de validación, aceptando tanto mejoras directas como, de manera probabilística, configuraciones subóptimas en etapas tempranas de la búsqueda. Este mecanismo permite escapar de óptimos locales y favorece una exploración más amplia del espacio de soluciones.

Al finalizar el proceso, el algoritmo conserva la configuración de hiperparámetros que produjo el mejor desempeño global, la cual se fija para la generación de resultados finales.

##### IV-C4. Generación y evaluación de las segmentaciones

Durante la fase de evaluación, el algoritmo se aplica de manera independiente a cada imagen del conjunto de datos. A partir de una máscara inicialmente vacía, el proceso recursivo va asignando etiquetas a las distintas regiones según las decisiones tomadas en cada nivel de la jerarquía quadtree. El resultado final es una máscara de segmentación completa, construida de forma adaptativa en función de la complejidad local de la imagen.

Las máscaras predichas se comparan posteriormente con las máscaras de referencia mediante las métricas definidas en el framework de evaluación, lo que permite analizar cuantitativamente el desempeño del método y contrastarlo con los enfoques basados en redes neuronales profundas.

## V. MODELOS DE CLASIFICACIÓN PARA SEGMENTACIÓN

En esta sección se describen los modelos de clasificación empleados como componentes fundamentales de las estrategias de segmentación guiada basadas en particionado espacial. A diferencia de los enfoques de segmentación directa, estos métodos delegan la toma de decisiones semánticas a clasificadores entrenados para discriminar entre zonas frágiles y dúctiles, cuya salida se utiliza posteriormente para construir máscaras de segmentación a distintas escalas.

El objetivo de esta sección es caracterizar los clasificadores utilizados, independientemente del esquema de particionado específico en el que se integran. De este modo, se separa explícitamente el análisis del modelo de clasificación del

mecanismo geométrico de segmentación, permitiendo evaluar de forma aislada el impacto de la arquitectura del clasificador en el desempeño final del sistema. Las subsecciones siguientes presentan los distintos modelos considerados.

#### V-A. Clasificador CNN

El clasificador basado en Redes Neuronales Convolucionales (CNN) fue diseñado para clasificar regiones de tamaño variable extraídas por el algoritmo Quadtree.

La arquitectura sigue un patrón de extracción de características seguido de clasificación:

$$\begin{aligned} \text{Entrada} &\rightarrow \text{Bloques Conv.} \rightarrow \text{Pooling Global} \\ &\rightarrow \text{Clasificador} \rightarrow \text{Softmax} \end{aligned} \quad (2)$$

Cada bloque convolucional aplica la siguiente secuencia:

1. Convolución 3×3 con padding=1
2. Batch Normalization
3. Activación ReLU
4. Convolución 3×3 con padding=1
5. Batch Normalization
6. Activación ReLU
7. Max Pooling 2×2 con stride=2

Se usa un kernel de 3 × 3 porque es el tamaño mínimo que captura información espacial (arriba, abajo, izquierda, derecha y diagonales). Al apilar múltiples capas con kernels pequeños se obtiene un campo receptivo grande con menos parámetros que usando un kernel grande directamente.

El padding de 1 píxel mantiene las dimensiones espaciales constantes después de cada convolución, lo que simplifica el diseño de la red.

Batch Normalization se incluye para acelerar el entrenamiento y estabilizar los gradientes, permitiendo usar tasas de aprendizaje más altas.

Se usan dos convoluciones por bloque antes del pooling para aumentar la capacidad de la red sin reducir las dimensiones espaciales prematuramente.

El parámetro `num_blocks` controla la profundidad de la red. Como cada bloque reduce las dimensiones a la mitad mediante Max Pooling, el tamaño mínimo de entrada es  $2^n \times 2^n$  píxeles, donde  $n$  es el número de bloques.

Cuadro I  
RELACIÓN ENTRE NÚMERO DE BLOQUES Y TAMAÑO MÍNIMO DE ENTRADA

| num_blocks | Tamaño mínimo | Canales finales |
|------------|---------------|-----------------|
| 2          | 4×4           | 64              |
| 3          | 8×8           | 128             |
| 4          | 16×16         | 256             |
| 5          | 32×32         | 512             |

El valor por defecto es 3 bloques, lo que permite clasificar regiones de 8×8 píxeles. Este tamaño fue elegido porque el Quadtree puede subdividir hasta regiones pequeñas, y 8×8 se consideró suficiente para contener información visual distinguible mientras permite segmentación detallada.

El número de filtros se duplica en cada bloque, comenzando desde 32:

$$\text{filtros en bloque } i = 32 \times 2^{i-1} \quad (3)$$

Esta progresión compensa la reducción de dimensiones espaciales: a medida que la imagen se hace más pequeña, se aumentan los canales para mantener la capacidad de representación.

Antes de la clasificación se aplica `AdaptiveAvgPool2d((1,1))`, que reduce cualquier tamaño espacial a 1×1 calculando el promedio de cada canal.

Esta capa permite que la red acepte entradas de cualquier tamaño (siempre que cumplan el mínimo). Sin importar si la entrada es 8×8 o 256×256, la salida siempre es un vector de tamaño fijo que puede procesarse por las capas densas.

La cabeza de clasificación consiste en:

1. Aplanamiento del tensor
2. Dropout (50 %)
3. Capa densa con reducción a 1/4 de los canales
4. Activación ReLU
5. Dropout (25 %)
6. Capa densa final a 3 clases

El Dropout previene sobreajuste, que es una preocupación con datasets pequeños de imágenes SEM. La capa intermedia reduce la dimensionalidad gradualmente antes de la clasificación final.

La red produce probabilidades mediante Softmax, donde la probabilidad más alta indica la clase predicha y su valor representa la confianza. Esta confianza es usada por el Quadtree para decidir si subdividir la región o aceptar la clasificación.

El entrenamiento usa Cross-Entropy Loss y el optimizador Adam. Las imágenes se agrupan por tamaño para formar batches, ya que las regiones del Quadtree tienen dimensiones variables y solo se pueden apilar en un tensor imágenes del mismo tamaño.

#### V-B. Clasificador ViT

El clasificador basado en Vision Transformer se utiliza como un componente auxiliar dentro del esquema de segmentación jerárquica, con el objetivo de asignar una única etiqueta a regiones completas de la imagen. A diferencia de los modelos de segmentación, este enfoque no opera a nivel de píxel, sino que produce una predicción global acompañada de una medida de confianza, la cual resulta fundamental para guiar las decisiones del algoritmo quadtree.

##### V-B1. Arquitectura del clasificador

La arquitectura adoptada corresponde a un Vision Transformer estándar para tareas de clasificación de imágenes, cuyo elemento distintivo es el uso de un token de clasificación. La imagen de entrada se divide inicialmente en parches de tamaño fijo, los cuales son proyectados a un espacio de características de alta dimensión mediante un proceso de incrustación. A estas representaciones se les añade información posicional aprendible, permitiendo al modelo preservar las relaciones espaciales entre los distintos parches.

Sobre esta secuencia de representaciones se introduce un token de clasificación, el cual no está asociado a ninguna región específica de la imagen. Este token se antepone a la secuencia de parches y participa activamente en los mecanismos

de autoatención del transformer. A lo largo de las capas del encoder, el token de clasificación actúa como un agregador de información global, concentrando en una única representación los patrones más relevantes presentes en la imagen completa.

Finalizado el procesamiento por el transformer, se descartan las representaciones asociadas a los parches y se conserva únicamente el vector correspondiente al token de clasificación. Este vector se proyecta mediante una cabeza de clasificación hacia el espacio de clases, generando puntuaciones que se transforman en probabilidades normalizadas. Estas probabilidades permiten obtener tanto la etiqueta predicha como una estimación explícita de la confianza del modelo.

#### V-B2. *Uso del clasificador en regiones de tamaño arbitrario*

Dado que el clasificador se emplea para evaluar regiones de distinto tamaño dentro del esquema quadtree, resulta necesario un preprocesamiento que garantice la compatibilidad con la entrada de tamaño fijo requerida por el Vision Transformer. Cada región extraída de la imagen original se normaliza en formato y número de canales, y posteriormente se redimensiona mediante interpolación a la resolución esperada por el modelo.

Este procedimiento permite aplicar un clasificador entrenado sobre imágenes completas a subregiones arbitrarias, manteniendo la coherencia de las predicciones y asegurando una integración transparente con el algoritmo de segmentación jerárquica.

#### V-B3. *Entrenamiento y rol dentro del framework*

El clasificador puede entrenarse de manera independiente sobre conjuntos de datos externos de clasificación, utilizando un esquema supervisado estándar. Una vez finalizado el entrenamiento, el modelo se emplea principalmente en modo de inferencia, proporcionando predicciones rápidas y consistentes durante la ejecución del algoritmo quadtree.

Dentro del framework AutoML, este clasificador actúa como un componente intercambiable, lo que permite evaluar distintas arquitecturas de clasificación bajo el mismo esquema de segmentación. De este modo, el análisis se centra en estudiar cómo la capacidad discriminativa del clasificador influye en la calidad final de la segmentación basada en descomposición jerárquica.

## VI. PIPELINE DE AUMENTACIÓN DE DATOS

Se implementó un conjunto completo de técnicas de aumentación de datos con el objetivo de incrementar la diversidad del dataset y mejorar la generalización de los modelos. La estrategia combinó transformaciones geométricas, fotométricas y específicas para SEM, aplicadas de manera secuencial y probabilística según un pipeline de aumentación.

### VI-1. *Aumentaciones Geométricas*

Afectan tanto a imágenes como a máscaras, manteniendo la alineación espacial:

- Rotación: Rota la imagen y la máscara dentro de un rango definido, simulando diferentes orientaciones de la muestra.
- Volteo: Invierte horizontal o verticalmente.
- Escalado: Realiza zoom in/out manteniendo el tamaño original.

- Traslación: Desplaza la imagen y máscara horizontal y/o verticalmente.
- Recorte aleatorio: Extrae subregiones aleatorias, redimensionadas al tamaño original.

Estas transformaciones permiten al modelo generalizar a variaciones espaciales de la muestra.

### VI-2. *Aumentaciones Fotométricas*

Modifican únicamente las imágenes, simulando variaciones en iluminación y ruido, sin afectar las máscaras:

- Brillo.
- Contraste.
- Corrección gamma.
- Ruido gaussiano.
- Desenfoque gaussiano.

Estas técnicas permiten al modelo aprender invariancia frente a condiciones de adquisición o ruido instrumental.

### VI-3. *Aumentaciones Específicas para SEM*

Diseñadas para capturar artefactos y características típicas de imágenes SEM:

- Deformación elástica (ElasticDeformationAugmentator): Simula variaciones naturales en texturas de roca o deformaciones durante la adquisición.
- Ecuilización adaptativa de histograma (AdaptiveHistogramEqualizationAugmentator): Mejora contraste local mediante CLAHE.
- Artefactos de carga: Añade manchas típicas de muestras no conductivas.
- Ruido de líneas de escaneo (ScanLineNoiseAugmentator): Simula líneas de escaneo o barrido que aparecen en SEM.

Estas técnicas aumentan la robustez del modelo frente a artefactos específicos de la microscopía electrónica.

### VI-4. *Composición de Aumentaciones*

Se implementaron métodos compuestos que permiten combinar varias aumentaciones para crear un pipeline complejo y controlado:

- Secuencial: Aplica múltiples aumentaciones de manera secuencial.
- Aplicación probabilística: Aplica una aumentación con una probabilidad determinada, introduciendo variabilidad controlada.
- Selección aleatoria: Elige una aumentación de un conjunto con probabilidades ponderadas.

Estas estrategias permiten generar un dataset enriquecido sin comprometer la coherencia entre imágenes y máscaras.

### VI-A. *Métricas de Evaluación*

La evaluación del desempeño de los modelos de segmentación se diseñó de forma modular y extensible, permitiendo calcular múltiples métricas a partir de una representación común de las predicciones y las máscaras de referencia. Todo el proceso es coordinado por un nodo evaluador central (*EvaluatorNode*), cuyo objetivo es desacoplar la generación de predicciones del cálculo de métricas, facilitando comparaciones consistentes entre modelos y configuraciones experimentales.

Para cada imagen del conjunto de validación, el modelo de segmentación genera una máscara predicha, la cual se empareja con su máscara real correspondiente. Este proceso produce una colección de pares  $(\hat{M}, M)$ , denominada *MaskPairs*. En escenarios con validación cruzada k-fold, estas colecciones se agrupan por pliegue, resultando en una estructura del tipo lista de listas; en el caso simple de train/validation split, existe un único conjunto de pares.

Esta estructura es la entrada principal del *EvaluatorNode*, que no calcula métricas directamente, sino que delega el cómputo a evaluadores especializados. Cada evaluador implementa una métrica específica y devuelve un único valor escalar, el cual es finalmente agregado en un diccionario de resultados que resume el desempeño del modelo.

La mayoría de las métricas utilizadas se fundamentan en interpretar la segmentación como un problema de clasificación a nivel de píxel. Para una clase dada, cada píxel de la máscara predicha se compara con la máscara real y se clasifica como verdadero positivo (TP), falso positivo (FP), falso negativo (FN) o verdadero negativo (TN). Los evaluadores acumulan estos conteos a lo largo de todas las imágenes antes de calcular la métrica final.

#### VI-A1. Intersección sobre Unión (IoU) y coeficiente Dice

El *Intersection over Union* (IoU) y el coeficiente *Dice* son las métricas principales para cuantificar el solapamiento entre predicción y referencia. El IoU se define como la razón entre la intersección y la unión de ambas máscaras, penalizando tanto falsas detecciones como omisiones. El Dice, estrechamente relacionado, pondera el solapamiento relativo y puede interpretarse como una forma del F1-score a nivel de píxel, siendo ligeramente más sensible a errores de clasificación.

Estas métricas se calculan de tres maneras complementarias: (i) por clase, evaluando individualmente cada etiqueta; (ii) promedio macro, donde se promedia el valor de cada clase sin ponderación, ofreciendo una visión balanceada incluso ante desbalances severos; (iii) promedio ponderado, donde cada clase contribuye según su frecuencia en el conjunto de datos, reflejando el desempeño global pero siendo sensible a clases dominantes como el fondo.

#### VI-A2. Precisión y Recall

La precisión y el recall permiten un diagnóstico más fino de los errores del modelo. La precisión mide la confiabilidad de las predicciones positivas, mientras que el recall cuantifica la capacidad del modelo para detectar todos los píxeles relevantes de una clase. Al igual que en IoU y Dice, se reportan promedios macro para analizar el comportamiento medio del modelo en todas las clases, independientemente de su tamaño relativo.

#### VI-A3. Cohesión de la máscara

Además de las métricas clásicas basadas en coincidencia píxel a píxel, se incorpora una métrica estructural denominada *Mask Cohesion*. Esta se implementa mediante un autoencoder convolucional entrenado exclusivamente con máscaras reales del conjunto de datos. El autoencoder aprende una representación compacta de las estructuras válidas presentes en las segmentaciones de referencia.

Durante la evaluación, las máscaras predichas se reconstruyen a través de este autoencoder y se calcula el error de

reconstrucción. Un error bajo indica que la predicción presenta estructuras coherentes y plausibles, similares a las observadas en las máscaras reales, mientras que un error alto sugiere ruido, fragmentación o patrones no realistas. Esta métrica complementa a IoU y Dice al capturar aspectos cualitativos de la segmentación que no siempre se reflejan en métricas puramente basadas en solapamiento.

Este conjunto de métricas proporciona una evaluación integral del desempeño, abarcando exactitud cuantitativa, balance entre clases y calidad estructural de las segmentaciones producidas.

## VII. RESULTADOS

## VIII. CONCLUSIONES